Sheth L.U.J. And Sir M.V. College

Aim: Implementation of nesC Programming Model
Create a basic nesC application using modules and configurations to understand component wiring and interface binding.

Step 1: The Module (`BlinkC.nc`)
This component contains only the code logic. It does not know where the timer or LEDs come from; it just declares that it needs them.
CODE:

```
#include "Timer.h"

module BlinkC {
  // Interfaces used by this module
  uses interface Boot;
  uses interface Leds;
  uses interface Timer<TMilli> as Timer0;
}
implementation {
  event void Boot.booted() {
    // Command: start timer
    call Timer0.startPeriodic(500);
  }

  event void Timer0.fired() {
    // Command: toggle LED
    call Leds.led0Toggle();
  }
}
```

MITHIL KADAM T083

Name :- Mithil Kadam
Roll No :- T083
Subject:- Wireless And Sensor Networks

Step 2: The Configuration / Wiring (`BlinkAppC.nc`)

This component has no logic. Its only job is component wiring (binding interfaces together).

Code:

```
configuration BlinkAppC {
}
implementation {
  // 1. Declare the components
  components MainC;
  components LedsC;
  components new TimerMilliC() as Timer0;
  components BlinkC as App;

  // 2. Wire the interfaces (Interface Binding)
  App.Boot   -> MainC.Boot;
  App.Leds   -> LedsC;
  App.Timer0 -> Timer0;
}
```

Step 3: The Runner Script (`sim.py`)

CODE:

```
import sys
from TOSSIM import *

t = Tossim([])
t.addChannel("LedsC", sys.stdout)

# Boot mote 1
m = t.getNode(1)
m.bootAtTime(0)

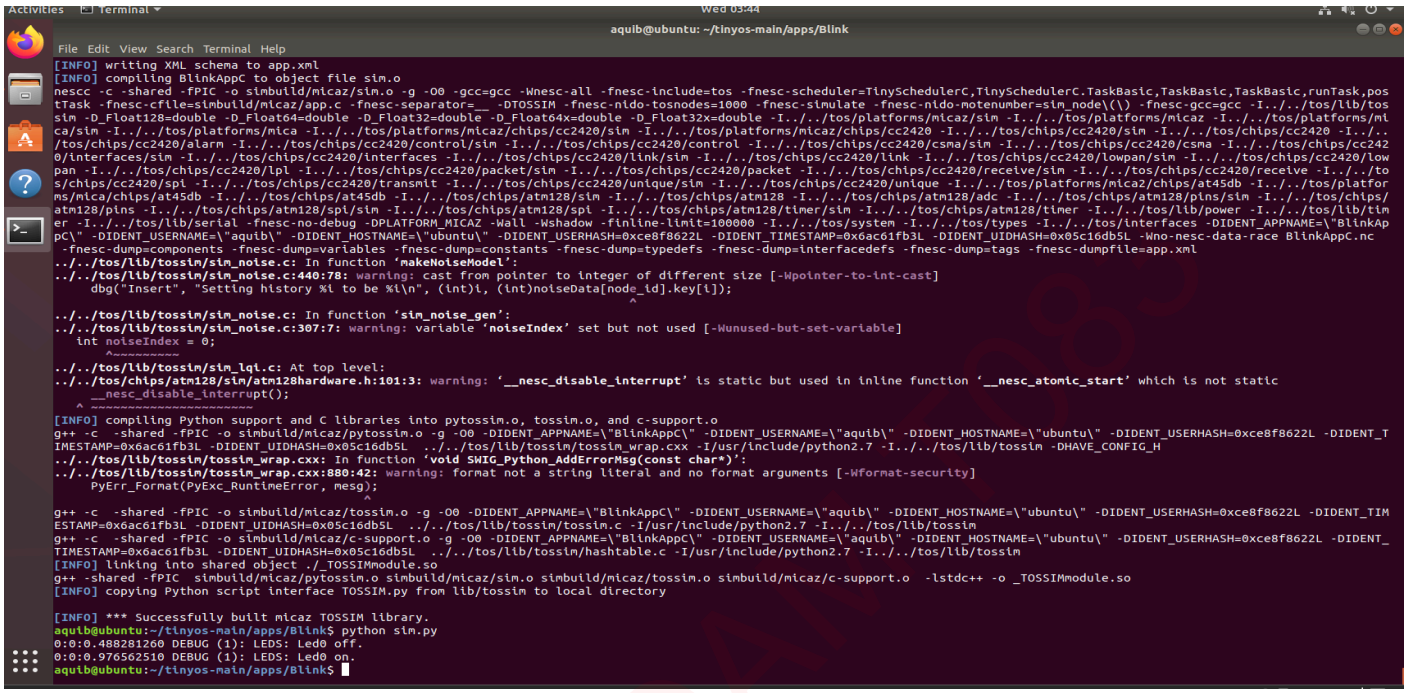
# Run 20 events
for i in range(20):
    t.runNextEvent()
```

MITHIL KADAM T083

Name :- Mithil Kadam
Roll No :- T083
Subject:- Wireless And Sensor Networks

Sheth L.U.J. And Sir M.V. College

OUTPUT:



Name :- Mithil Kadam
Roll No :- T083
Subject:- Wireless And Sensor Networks